

Intern Assignment

Product Intelligence Dashboard for E-commerce Sellers

Build a deployed product that extracts product data, validates listing quality, compares competitor prices, and raises actionable alerts.

Assignment Summary

You will build an end-to-end web application with a frontend UI, backend APIs, job processing, dashboards, alerts, and deployment. The app should work with product video input first, and support product CSV upload as a fallback.

Role	Software / Product Engineering Intern
Recommended Time	48 to 72 hours
Expected Output	Deployed frontend, deployed backend, GitHub repo, README, sample inputs, and demo notes
Important Note	Mocked or simulated integrations are acceptable where real integrations are difficult.

1. Background

Assume we are a seller on Flipkart. We sell products online and want to monitor listing quality, competitor pricing, and product improvement opportunities. Today, a seller often has product videos, product feeds, marketplace listings, and competitor price signals, but no simple way to combine them into one actionable dashboard.

Your task is to build a deployed web application that helps the seller understand whether a product listing is good enough, whether the product price is competitive, and what action should be taken next.

2. Objective

Build an end-to-end Product Intelligence Dashboard where a seller can upload a product video or product CSV, extract or enter product information, validate the listing, optionally generate enhanced titles using product attributes and trending keywords, compare prices against competitors, and receive alerts for critical issues.

What we are testing

We are not only testing coding syntax. We are testing your ability to figure out ambiguous requirements, design a practical solution, build reliable APIs, create a usable UI, handle edge cases, and deploy the project end to end.

3. Expected User Flow

1. User opens the web app and uploads a short product video.
2. User selects whether title enhancement should be enabled.
3. System creates a processing job and shows job status.
4. System extracts or simulates product information from the video.
5. If video extraction is incomplete, user can upload a product CSV or manually edit product details.
6. System validates product data and identifies listing issues.
7. System generates enhanced title suggestions if title enhancement is enabled.

8. System uploads, simulates, or fetches competitor prices for similar products across e-commerce platforms.
9. Dashboard shows product quality, competitor price comparison, and alerts.
10. User can refresh competitor prices and see updated alerts.

4. Inputs

4.1 Primary Input: Product Video

The user should be able to upload a short product video. From the video, the system should try to extract product details such as product name, brand, category, color, material, visible packaging text, and possible attributes.

- Frame extraction from the video
- OCR on selected frames
- Image/video analysis using an AI model or API
- A mocked extraction layer with clear assumptions
- Manual review or edit flow after extraction

Accuracy does not need to be perfect. The goal is to show a practical working product flow.

4.2 Fallback Input: Product Feed CSV

If product information cannot be extracted properly from the video, the user should be able to upload a product CSV or manually edit the extracted data.

Column 1	Column 2	Column 3	Column 4
sku_id	product_title	description	brand
category	price	mrp	image_url
product_url	availability	color	size
material			

4.3 Title Enhancement Flag

The upload form should include a boolean flag: Enhance product title? Yes / No.

If the flag is set to Yes, the system should extract useful product attributes, identify relevant keywords, and generate an improved product title.

Original title	Nike Shoes Blue
Extracted attributes	Brand: Nike; Color: Blue; Product type: Running Shoes; Gender: Men; Material: Mesh
Suggested keywords	running shoes, lightweight shoes, sports shoes for men
Enhanced title	Nike Blue Lightweight Running Shoes for Men with Mesh Upper

5. Core Requirements

Area	What to Build	Minimum Expectation
Product Information Extraction	Process uploaded product video and extract or simulate product details. Show extraction status, missing fields, and allow fallback CSV/manual input.	Working implementation with clear assumptions; mocks are allowed where appropriate.
Product Feed Validation	Validate product data and detect issues such as missing title, weak description, invalid price, missing image, duplicate SKU, missing attributes, or out-of-stock status.	Working implementation with clear assumptions; mocks are allowed where appropriate.

Product Quality Dashboard	Show total products processed, issue counts by severity, weak listings, missing image count, invalid price count, and overall listing quality score.	Working implementation with clear assumptions; mocks are allowed where appropriate.
Enhanced Title Suggestions	When the title enhancement flag is enabled, generate improved titles using attributes and relevant keywords. Show original title, attributes, keywords, enhanced title, and reason.	Working implementation with clear assumptions; mocks are allowed where appropriate.
Competitor Price Input or Simulation	Assume we are selling on Flipkart. Show competitor prices from platforms such as Amazon, Myntra, Ajio, Nykaa Fashion, Tata Cliq, or Meesho using mock data, CSV upload, manual input, or safe APIs.	Working implementation with clear assumptions; mocks are allowed where appropriate.
Price Comparison Dashboard	Compare our Flipkart price with competitor prices. Show lowest, highest, average competitor price, price gap, percentage difference, and recommended action.	Working implementation with clear assumptions; mocks are allowed where appropriate.
Alerts and Notifications	Raise alerts for critical listing and pricing issues. In-app alert history is acceptable. Real email, Slack, Telegram, or WhatsApp notification is a bonus.	Working implementation with clear assumptions; mocks are allowed where appropriate.
Scheduled or Simulated Price Refresh	Support competitor price refresh. A real scheduler is a bonus; a manual Refresh Prices button with backend logic is acceptable for the minimum version.	Working implementation with clear assumptions; mocks are allowed where appropriate.
Job Status Tracking	Long-running operations should be tracked as jobs with statuses such as PENDING, RUNNING, COMPLETED, FAILED, and PARTIALLY_COMPLETED.	Working implementation with clear assumptions; mocks are allowed where appropriate.
Deployment	Deploy both frontend and backend so reviewers can test the project without running it locally.	Working implementation with clear assumptions; mocks are allowed where appropriate.

6. Product Validation Rules

Your system should detect listing issues and classify them by severity. You may add your own rules, but at minimum include a meaningful subset of the following.

Issue Type	Severity Example	Suggested Fix Example
Missing title	HIGH	Add a clear product title.
Very short title	MEDIUM	Add brand, product type, color, gender, or material.
Missing brand	MEDIUM	Add brand if known, or mark as unbranded.
Invalid price	HIGH	Price should be positive and numeric.
MRP lower than selling price	HIGH	Correct MRP or selling price.
Missing image	HIGH	Add at least one product image.
Broken image URL	MEDIUM	Replace with an accessible image URL.
Duplicate SKU	HIGH	Keep SKU IDs unique.
Weak description	LOW	Add more product details and attributes.
Missing important attributes	MEDIUM	Add color, size, material, gender, or category-specific attributes.
Out of stock product	LOW	Mark separately or notify operations team.

7. Competitor Pricing Scope

Assume our own selling platform is Flipkart. The system should compare our product price against competitor prices from other e-commerce platforms. Do not make live scraping mandatory. If you use scraping, only do it where legally and technically allowed. Do not bypass bot protection or platform security controls.

Data Source Option	Accepted?	Notes	Expected Output
Mock competitor data	Yes	Recommended for fastest reliable implementation.	Price comparison should still work.
Competitor price CSV upload	Yes	Good for demonstrating data ingestion.	Store and compare prices.
Manual competitor price entry	Yes	Acceptable for minimum version.	Show price history and alerts.
Public APIs	Yes	Use only if easy and reliable.	Document API assumptions.
Live scraping	Optional	Not required and should not bypass restrictions.	Use only safe and allowed sources.

8. Alerts and Notification Rules

The system should create alerts when product quality or competitor pricing requires attention. An in-app notification log is acceptable for the minimum version. Real external notifications are a bonus.

- Create a HIGH severity alert if a product has no title, no price, or invalid price.
- Create a HIGH severity alert if our Flipkart price is more than 10 percent higher than the lowest competitor price.
- Create a MEDIUM severity alert if the title is too weak or important attributes are missing.
- Create a MEDIUM severity alert if a competitor price drops significantly during refresh.
- Create a LOW severity alert for weak descriptions or out-of-stock products.

9. Job Status Tracking

Since video processing, CSV validation, and competitor price refresh can take time, the application should create jobs and expose progress in the UI.

Job ID	Status	Progress	Timestamps	Error Handling
Unique ID	PENDING / RUNNING / COMPLETED / FAILED / PARTIALLY_COMPLETED	0 to 100 percent	started_at, completed_at	Clear error message and failed row details where possible

10. Expected UI Screens

- Upload screen for product video, CSV fallback, and title enhancement flag.
- Processing screen or jobs list showing job status and progress.
- Product quality dashboard showing summary metrics and issue counts.
- Product list table with filters for severity, category, and alert status.
- Product detail page showing extracted data, listing issues, enhanced title, and competitor prices.
- Competitor price comparison section with price gap and recommended action.
- Alerts or notification history page.
- Refresh competitor prices action.

11. Suggested Backend APIs

You can design your own APIs. The following structure is suggested for clarity.

Method	Endpoint	Purpose
POST	/upload-video	Upload product video and create processing job.
POST	/upload-products-csv	Upload fallback product feed CSV.
GET	/jobs	List jobs.
GET	/jobs/{job_id}	Get job status and progress.
GET	/products	List processed products.
GET	/products/{sku_id}	Get product details.
GET	/dashboard/quality-summary	Get quality dashboard summary.
GET	/products/{sku_id}/issues	Get product-level issues.
POST	/products/{sku_id}/enhance-title	Generate enhanced title for one product.
POST	/competitor-prices/upload	Upload competitor price CSV.
POST	/competitor-prices/refresh	Refresh or simulate price updates.
GET	/products/{sku_id}/competitor-prices	Get competitor prices for a product.
GET	/alerts	List alerts.
POST	/alerts/rules	Create or update alert rules, optional.

12. Minimum Required Features

1. Upload product video.
2. Extract or simulate product information from video.
3. Support fallback product CSV upload or manual edit.
4. Validate product data and detect listing issues.
5. Show product quality dashboard.
6. Add title enhancement flag.
7. Generate enhanced title suggestions when the flag is enabled.
8. Upload, manually enter, or simulate competitor prices.
9. Compare our Flipkart price with competitor prices.
10. Raise alerts for critical listing or pricing issues.
11. Show job status while processing.
12. Deploy frontend and backend.
13. Submit GitHub repo and README.

13. Bonus Features

- Real OCR from video frames.
- Real AI-based product information extraction.
- Real email, Slack, Telegram, or WhatsApp notification.
- Scheduled background price refresh.
- Price history chart.
- Downloadable product quality report.
- Authentication.
- Retry failed jobs.
- Product-level recommendation engine.
- Better keyword trend matching.

- Clean API documentation or Swagger/OpenAPI docs.
- Dockerized deployment.

14. Deliverables

1. Deployed frontend URL.
2. Deployed backend API URL.
3. GitHub repository link.
4. README with setup and usage instructions.
5. Sample product video or sample product CSV.
6. Sample competitor price CSV or mock data file.
7. Short architecture explanation.
8. Known limitations and assumptions.
9. Screenshots or short demo video.

15. README Requirements

- Tech stack used.
- How to run locally.
- How to use the deployed app.
- API documentation.
- Data model or schema explanation.
- Deployment links.
- Assumptions made.
- What is real vs mocked.
- Trade-offs and limitations.
- What you would improve with more time.

16. Evaluation Criteria

Area	Weight	What We Look For
End-to-end completion	25%	The app works from input to dashboard to alerts, and is deployed.
Backend API and data modeling	20%	Clean API structure, sensible schema, job handling, error handling.
Frontend usability	15%	Clear screens, easy review flow, useful product detail views.
Validation and business logic	20%	Meaningful listing checks, issue severity, title enhancement, recommendations.
Competitor pricing and alerting	10%	Useful price comparison, refresh flow, alert rules, price history if available.
Deployment and README quality	10%	Reviewer can open the app, understand the project, and run it locally if needed.

17. Submission Instructions

Please submit your assignment by sharing the following in one message or email:

- Frontend deployment link.
- Backend API link.

- GitHub repository link.
- README link or README included in the repository.
- Any credentials or test instructions needed to review the app.
- Sample files used for testing.
- A short note explaining what is fully implemented, what is mocked, and what you would improve next.

Fair implementation guidance

Use mocks where real integrations are time-consuming or unreliable. We value a complete, reliable, well-explained product flow more than a partially working integration with fragile external dependencies.

Appendix A: Sample Product CSV Format

sku_id	product_title	brand	category	price	mrp	availability
SHOE001	Nike Blue Running Shoes	Nike	Shoes	3999	4999	in_stock
DRESS001	Red Dress	Zara	Dresses	1799	2499	in_stock
BAG001		Puma	Bags	abc	1299	out_of_stock

Additional optional columns: description, image_url, product_url, color, size, material.

Appendix B: Sample Competitor Price CSV Format

sku_id	product_name	platform	competitor_url	competitor_price	currency	last_checked_at
SHOE001	Nike Blue Running Shoes	Amazon	https://example.com/a	3499	INR	2026-05-22
SHOE001	Nike Blue Running Shoes	Myntra	https://example.com/m	3799	INR	2026-05-22
DRESS001	Red Dress	Ajio	https://example.com/j	1599	INR	2026-05-22

Appendix C: Candidate Notes

- You are free to choose any frontend and backend technology stack.
- You are free to use SQL, NoSQL, files, or in-memory storage for the demo, but persistence is preferred.
- You should document all assumptions clearly.
- Do not spend time bypassing website protections. Use safe data sources or mocks for competitor prices.
- Focus on making the reviewer experience smooth and easy to understand.